

# TP1 Integration and Regression Plan

Merge schedule, regression checkpoints, and the rules that keep the documents and the code in agreement

**Role:** Design Documents and Architecture — integration coordination and shared infrastructure  
**Author:** A.G. (agupt545)  
**Version:** 1.00    **Date:** 2026-09-17  
**Repository:** `github.com/adityagupta11-debug/cse360`, branch `main`, folder `TP1/`

## 1. What this role owns

This role does not own a feature. It owns the parts of the project that every feature touches: the shared infrastructure in `database.Database` and the validation layer, the architecture and detailed design diagrams, the traceability matrix, and the schedule on which the team merges and re-runs the tests. When two feature owners need the same method, this role writes it once and both build on it, rather than each writing a version and discovering the difference at integration.

## 2. Running the regression suite

The suite is a single class that runs every test driver in turn and prints one summary. It exits with a non-zero status when anything fails, so it can be run from a script as well as from Eclipse.

| How             | Command                                                                                                                                                                                     |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| From Eclipse    | Run <code>testingAutomation.TP1RegressionSuite</code> as a Java Application. The JavaFX and H2 user libraries are already on the project's build path.                                      |
| From a terminal | <pre>javac -d bin --module-path &lt;javafx&gt;/lib:&lt;h2.jar&gt; \$(find src -name "*.java") java -cp "bin:&lt;h2.jar&gt;:&lt;javafx&gt;/lib/*" testingAutomation.TP1RegressionSuite</pre> |
| Individually    | Each of the six test classes has its own <code>main</code> and can be run alone while working on one feature.                                                                               |

The suite uses private in-memory H2 databases (`jdbc:h2:mem:...`), one per test class, so running it never alters the `~/FoundationDatabase` file the application itself uses. A team member can run the tests and then demonstrate the application without first having to reset anything.

## 3. Merge and regression checkpoint schedule

Standups are Monday and Thursday. Integration happens on the same days, immediately after the standup, so that what was just discussed is what gets merged. A checkpoint is a full run of the regression suite plus a manual walk of the pages the merge touched.

| Checkpoint | When                  | What is merged                                                                                                                                       | Exit condition                                                                                                               |
|------------|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| C1         | Week 1, Thursday      | Shared infrastructure only: the new <code>Database</code> methods, <code>entityClasses.Invitation</code> , and <code>EmailAddressRecognizer</code> . | Suite green; the interfaces in §3.1 of the design document are published to the team so feature work can start against them. |
| C2         | Week 2, Monday        | First feature pages against the shared infrastructure.                                                                                               | Suite green; each merged feature has at least one test class.                                                                |
| C3         | Week 2, Thursday      | Remaining feature pages; first full pass of the traceability matrix.                                                                                 | Suite green; no user story row has an empty Test column.                                                                     |
| C4         | Week 3, Monday        | User interface pass: every new page takes the look and feel agreed in <i>User Experience.pdf</i> .                                                   | Suite green; every page builds and renders; no page still shows a “not yet implemented” alert.                               |
| C5         | Week 3, Thursday      | Documentation freeze: diagrams, matrix, and Javadoc brought level with the code.                                                                     | Suite green; diagrams regenerated from the sources; matrix reviewed line by line.                                            |
| C6         | Submission day, 12:00 | Nothing new. The archive is built from what is already merged and submitted as the safety copy.                                                      | Suite green from a clean checkout; the ZIP opens and the screencasts play.                                                   |

## 4. Rules for a merge

- Run the suite before you push, not after.** A push that leaves the suite red blocks everyone else's integration, which costs far more time than the run does.

2. **A feature arrives with its tests.** A merge that adds behavior and no test cases is not ready, because nothing will catch it when someone else changes the code underneath it.
3. **The feature owner updates the diagrams their change affects.** If a change adds a class, a package, or a method that appears in Figure 2, the PlantUML source is edited in the same merge and the PNG regenerated. The person who made the change is the only one who knows what it was.
4. **Shared infrastructure changes are raised at a standup first.** Anything in `database.Database`, the validation layer, or `entityClasses` is used by more than one person; changing it quietly is how two people's work stops fitting together.
5. **One package per feature.** New work goes in its own MVC package wherever it can. This is what has kept merge conflicts in this phase to the shared files only.
6. **The traceability matrix is updated with the code, not at the end.** A row written three weeks later is a guess; a row written with the merge is a record.

## 5. Known integration risks and how they are handled

| Risk                                                                     | Why it matters                                                                                                                                                                                        | How it is handled                                                                                                                                                                     |
|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Two feature owners change <code>Database.java</code> in the same window. | It is the one file nearly every feature needs, so it is where conflicts concentrate.                                                                                                                  | New methods are appended in the section for their feature rather than interleaved, and shared changes land at C1 before feature work starts.                                          |
| An existing <code>FoundationDatabase</code> file lacks the new columns.  | A team member who ran an earlier build has a database file from before this phase; a plain <code>CREATE TABLE IF NOT EXISTS</code> would leave it without the new columns and every query would fail. | <code>createTables</code> issues <code>ALTER TABLE ... ADD COLUMN IF NOT EXISTS</code> for every column added this phase, so an old file is migrated on the next start.               |
| The password evaluator prints a line per character examined.             | Useful when demonstrating the dynamic interface, but it buries a test report.                                                                                                                         | Test drivers that call the evaluator in a loop redirect <code>System.out</code> for that loop only, then restore it.                                                                  |
| A policy rule gets copied instead of reused.                             | Two copies drift, and the one nobody remembers stops being enforced.                                                                                                                                  | The one-time password deadline calls <code>ControllerAdminHome.validateInvitationDeadline</code> ; the account update page calls the same recognizes the account creation pages call. |
| Diagrams fall behind the code.                                           | A diagram that no longer describes the system is worse than none, because people rely on it.                                                                                                          | Rule 3 above, checked at C5, with the sources kept in the repository so regenerating is one command.                                                                                  |

## 6. Defects found and fixed during integration

These are recorded here because they were found by integration work rather than by a feature owner, and they belong in *TP1 Plan.pdf* as unplanned effort.

| # | Defect                                                                                                                                                                                                  | Effect                                                                                                                                | Fix                                                                                                                                                  |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <code>ControllerNewAccount</code> removed the invitation using <code>ViewNewAccount.text_Invitation.getText()</code> , a field that is never placed on the page and so always returned an empty string. | The invitation was never removed after an account was created, so one code could be used repeatedly to create any number of accounts. | Removal now uses <code>ViewNewAccount.theInvitationCode</code> , the code actually used. The unused field was deleted so no one repeats the mistake. |
| 2 | <code>ViewUserUpdate</code> displayed the stored password in clear text on the account page.                                                                                                            | Anyone who could see the screen could read the password.                                                                              | The page now shows a masked placeholder and reports only whether a password is set.                                                                  |
| 3 | <code>PasswordPopupWindow.show()</code> returned the text left in the field even when the user closed the window without pressing Finish.                                                               | Closing the window could store a partial, unvalidated password.                                                                       | The window records whether Finish was pressed and returns an empty string otherwise; the field is cleared on every use.                              |
| 4 | The account update page wrote whatever was typed into a dialog straight to the database.                                                                                                                | Any text, of any length, could be stored as a name, a <code>UserName</code> , or an email address.                                    | Each field is validated first by <code>ControllerUserUpdate</code> , and the user is told what was wrong.                                            |
| 5 | The password evaluator walked a very long string one character at a time before rejecting it.                                                                                                           | A long string pasted into the field produced thousands of lines of output and needless work.                                          | A size check now precedes the directed graph, matching the pattern used by the other recognizers.                                                    |